DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation

Yuhang Lai* [1], Chengxi Li* [1], Yiming Wang* [1,2], Tianyi Zhang* [3], Ruiqi Zhong* [4], Luke Zettlemoyer[5,6], Wen-tau Yih[6], Daniel Fried[7], Sida Wang[6], Tao Yu[1,5]

[1]The University of Hong Kong, [2]Peking University, [3]Stanford University, [4]UC Berkeley, [5]University of Washington, [6]Meta AI, [7]Carnegie Mellon University

* Indicates equal contribution

An Example Problem in DS-1000

Problem

Here is a sample dataframe:

```
df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
```

I'd like to add inverses of each existing column to the dataframe and name them based on existing column names with a prefix, e.g. inv_A is an inverse of column A and so on.

The resulting dataframe should look like so:

```
result = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6], "inv_A": [1/1, 1/2, 1/3], "inv_B": [1/4, 1/5, 1/6]})
```

Obviously there are redundant methods like doing this in a loop, ***but** there should exist much more pythonic ways of doing it* … [omitted for brevity]

Code Context

```
A:
<code>
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3],"B": [4, 5, 6]})
</code>
BEGIN SOLUTION
<code>
[insert]
</code>
END SOLUTION
<code>
print(result)
</code>
```

Reference Solution

```
result = df.join(df.apply(lambda x: 1/x).add_prefix("inv_"))
```

Prompt → Language Models (GPT-3 Codex) → Predict

Replace `[insert]` in the code context with following predicted code snippets

```
result = df.div(1).add_prefix("inv_")
```

→ Execute to evaluate

Multi-criteria Execution-based Evaluation

Test case 1

```
df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
ans = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6],
                    "inv_A": [1/1, 1/2, 1/3],
                    "inv_B": [1/4, 1/5, 1/6]})
```

Test case 2

```
df,ans = ...[omit for brevity]

pd.testing.assert_frame_equal(result, ans)
```

Surface-form constraints

`for` and `while` should not appear in Syntax Tree

→ Correct/wrong?

The Pipeline for Building DS-1000

❶ Manually Selecting and Modifying StackOverflow Problems

Here is a sample dataframe:

```
df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
```

I'd like to add inverses of each existing column to the dataframe and … [omitted for brevity]

try:

```
inv_df = df.join(df.apply(lambda x: 1/x).add_prefix("inv_"))
```

- ✓ High vote
- ✓ Testable
- ✓ Useful
- ✓ Representative

❷ Adding Code Context

```
import pandas as pd
df = pd.DataFrame({"A": [1, 2, 3],
                   "B": [4, 5, 6]})
### BEGIN SOLUTION
[insert]
### END SOLUTION
print(result)
```

❸ Implementing Automatic Tests

Test cases

…[omit for brevity]

```
pd.testing.assert_frame_equal(result, ans)
```

Surface-form constraints

`for` and `while` should not appear in Syntax Tree

❹ Perturbing Original Problem

*… I'd like to apply the **exponential** function to each existing column … The resulting dataframe should look like so:*

```
result = pd.DataFrame({"A": [1, 2, 3],
                       "B": [4, 5, 6],
                       "exp_A": [e^1, e^2, e^3],
                       "exp_B": [e^4, e^5, e^6]})
```

… [omitted for brevity]

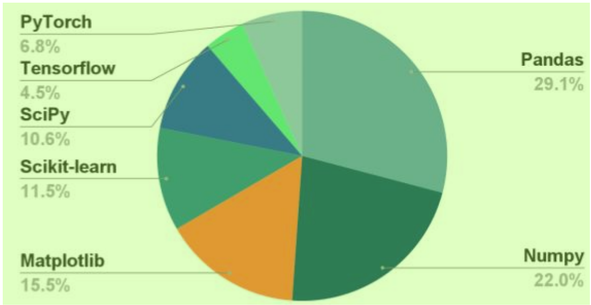
❺ Red Teaming

```
df = pd.DataFrame({"A": [1, 2, 3],
                   "B": [4, 5, 6]})
### BEGIN SOLUTION
# A known WRONG SOLUTION
result = df.join(df.apply(lambda
x:math.e**x).add_prefix('exp_'))
### END SOLUTION
print(result)
```

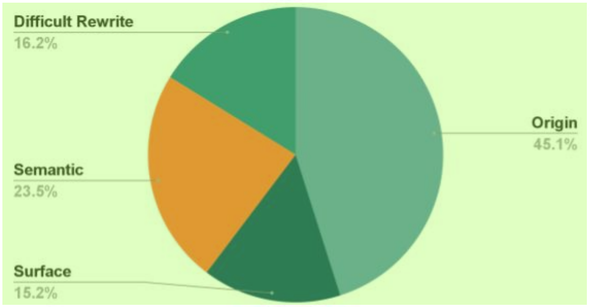
- Problems reflect ***diverse, realistic, and practical daily usage*** collected from StackOverflow
- Highly specific evaluation ensure its reliability - ***very few (1.8%)*** incorrect predicted solutions pass our evaluation
- Proactively ***defend against memorization*** by perturbations

Dataset Statistics

Data-Science Libraries



Different Perturbation Types



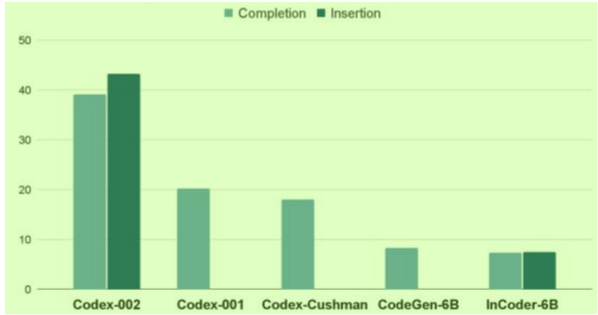
Comparison to other benchmarks - the first three target **general Python usage** and the next three involve **data science code generation**

Dataset	Problems	Evaluation	Avg. Test Cases	Avg. P Words	Avg. Lines of Code Solution	Data Source
HumanEval	164	Test Cases	7.7	23.0	6.3	Hand-Written
MBPP	974	Test Cases	3.0	15.7	6.7	Hand-Written
APPS	10000	Test Cases	13.2	293.2	18.0	Competitions
JuICe	1981	Exact Match + BLEU	-	57.2	3.3	Notebooks
DSP	1119	Test Cases	2.1	71.9	4.5	Notebooks
CoNaLa	2879	BLEU	-	13.8	1.1	StackOverflow
DS-1000	1000	Test Cases + Surface-Form Constraints	1.6	140.0	3.6	StackOverflow

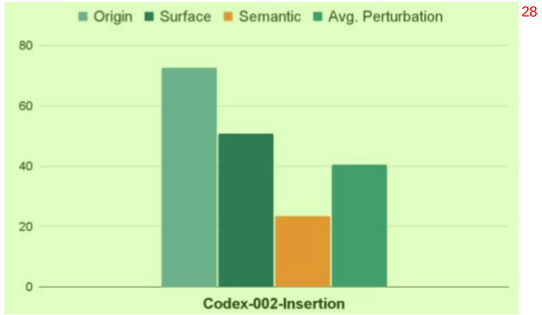
Experiments

A frequently occurring NumPy problem set on GitHub

Overall Pass@1 Accuracy With 40 Samples



Pass@1 Accuracy on Numpy-100



	Pandas	NumPy	Scikit-learn	SciPy	TensorFlow	PyTorch	Overall
$\text{Origin}_{\text{surface}}$	37.3	61.2	52.6	33.0	64.9	64.8	53.2
Surface	31.9 −5.4	58.4 −2.8	55.7 +3.1	32.1 −0.9	58.0 −8.9	50.0 −14.8	49.8 −3.4
$\text{Origin}_{\text{semantic}}$	36.8	56.7	60.6*	40.3	71.3	65.1	47.2
Semantic	33.2 −3.6	49.0 −7.7	38.9* −21.7	34.3 −6.0	42.5 −25.8	30.5 −34.6	38.2 −9.0
$\text{Origin}_{\text{difficult}}$	39.9	52.7	5.0*	58.1	73.0*	53.8*	46.8
Difficult Rewrite	17.7 −22.2	27.1 −25.6	0.0* −5.0	13.8 −44.3	38.0* −35.0	28.8* −25.0	21.0 −25.8